AI505

Optimization

Discrete Optimization
Randomized Optimization Heuristics

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

Outline

1. Randomized Optimization Heuristics

2. Search Methods

3. Constructive Search

Outline

Heuristics

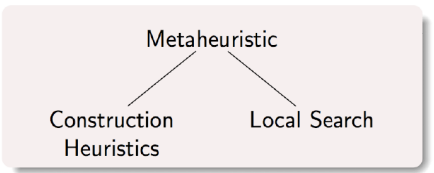
- Get inspired by approach to problem solving in human mind
  [A. Newell and H.A. Simon. “Computer science as empirical inquiry: symbols and search.” Communications of the ACM, ACM, 1976, 19(3)]
  - effective rules without theoretical support
  - trial and error
- Applications:
  - Optimization
  - But also in Psychology, Economics, Management [Tversky, A.; Kahneman, D. (1974). "Judgment under uncertainty: Heuristics and biases". Science 185]
- Basis on empirical evidence rather than mathematical logic. Getting things done in the given time.

Randomized Optimization Heuristics (ROHs)

Two main search paradigms:

- Constructive search (works on partial solutions)
- Local search (works on complete solutions)

plus high level guiding heuristics (ie, metaheuristics), eg, evolutionary algorithms.



Toy Examples

We will use the following examples to illustrate concepts:

- Knapsack Problem
- Traveling Salesman Problem
- Single Machine Total Weighted Tardiness
- Graph Vertex-Coloring

Knapsack Problem

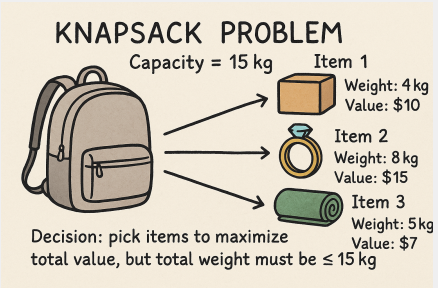
Given: A ground set of items with weights and values. A knapsack with a weight capacity.

Task: Find the subset that maximizes the total value

Example (Instance)

Capacity = 15

Item	Weight	Value
1	4	10
2	8	15
3	5	7
4	5	8
5	9	10
6	7	7
7	3	4
8	1	3

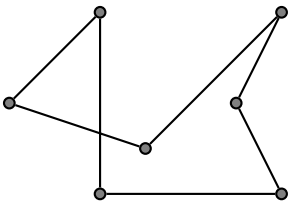


Sol. representation: Set: $\{1, 2, 7\}$ or Incidence vector: $[1, 1, 0, 0, 0, 0, 1, 0]$
Total weight: 15; Total value: 26

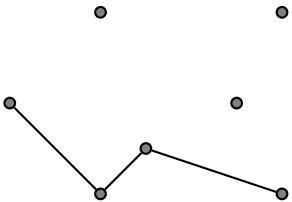
Traveling Salesman Problem

Given: A graph $G = (V, E)$ and a weight function $\omega : E \to \mathbb{R}$.

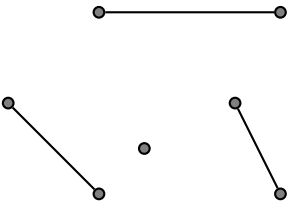
Task: Find the shortest Hamiltonian tour.



sol. representation:
permutation of vertices or
assignment of successors



sol. representation:
set of adjacent edges



sol. representation:
set of edges or
incidence binary vector

Single Machine Total Weighted Tardiness

Given: a set of n jobs $\{1, \ldots, n\}$ to be processed on a single machine and for each job j a processing time p_j, a weight w_j and a due date d_j.

Task: Find a schedule that minimizes the total weighted tardiness $\sum_{j=1}^{n} w_j \cdot T_j$, where $T_j = \max\{C_j - d_j, 0\}$ (C_j completion time of job j)

Example (Instance)

Job	1	2	3	4	5	6
Processing Time	3	2	2	3	4	3
Due date	6	13	4	9	7	17
Weight	2	3	1	5	1	2

Sol. representation:
Linear permutation
$\phi = [3, 1, 5, 4, 1, 6]$

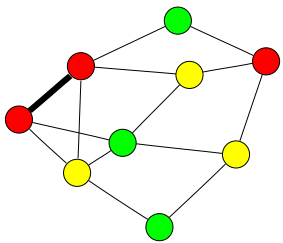
Job	3	1	5	4	2	6
C_j	2	5	9	12	14	17
T_j	0	0	2	3	1	0
$w_j \cdot T_j$	0	0	2	15	3	0

Graph Vertex-Coloring

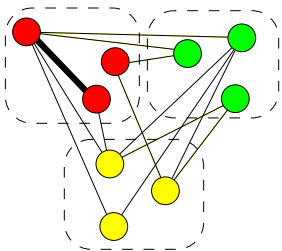
Given: A graph G and a set of colors Γ.

A **proper coloring**: each vertex receives a color and no two adjacent vertices receive the same color.

Task:
Find a proper coloring of G that uses the minimal number of colors (chromatic number).



Assignment representation



Partitioning representation

Outline

A Search Problem

Problem statement

Constrained Optimization Problem: $\min\{f(s) \mid s \in \mathcal{F}\}$

- $\mathcal{F} \subseteq \mathcal{S}$ set of **feasible solutions**
- $\mathcal{S}$ set of **candidate solutions** (**combinatorial structures**)

The concept of **feasibility** is flexible and a design choice.
Most typically, it implies satisfying the constraints of the problem.

Guiding rule: if it has an **objective function value**, then it is a **feasible solution**

Guiding rule: **constructive search algorithms** work with **partial, infeasible solutions**

Example: a partial tour does not have an objective function value (but knapsack partial solutions do have an objective function value)

Constraint Handling

Constraints in heuristic methods are handled:

- **implicitly** in the definition of the **combinatorial structures** that constitute the search states: assignments, permutations, (sub)sets, partitions, (sub)graphs, sequences, set of sequences, ...
- as **one-way** constraints between variables
- as **soft constraints**
  ie, relaxed in the evaluation function as penalty components with large weights or as lexicographically more important componenents

ROHs: Reasoning, Metaheuristics

- Stochastic Local Ssearch
- Simulated Annealing
- Iterated Local Search
- Tabu Search
- Variable Neighborhood Search
- Adaptive Large Neighborhood Search
- Evolutionary Algorithms
- Ant Colony Optimization
- Estimation-of-Distribution Algorithms
- Artificial Immune Systems
- ...
- Evolutionary Computation Bestiary `http://fcampelo.github.io/EC-Bestiary/`
- **Super**natural inspired [Maturana, Fouhey, 2013]

A Classification

- White box optimization:
  models can be expressed mathematically

- Grey box optimization:
  internal information about objective function computation is often available
  models that have a mathematical expression but may need data to determine them (eg, neural networks)

- Black box optimization:
  no mathematical expression is available

Approaches to ROHs

- White/Grey box: representation (modelling) + reasoning (search)
  constraint based local search, comet, local solver (Hexaly)

- Black Box: a different approach, framework separating problem from solvers and defining the interface specification

$\Longrightarrow$ COST Action: ROAR-NET

https://roar-net.eu

A Search Problem

Definition (Search or Optimization Algorithm)

Goal formulation: we want to find the minimum with respect to some criterion from a set of candidate elements.

Problem formulation: Given a description of the states, an initial state and actions necessary to reach the goal, find a sequence of actions to reach the goal.

Search: the algorithm simulates sequences of actions in the model of the goal, searching until it finds a sequence of actions that reaches the goal.
The algorithm might have to simulate multiple tentative answers that do not meet the goal, but eventually it reaches a solution, or it will find that no solution is possible.

Components of a Search Algorithm (1)

(valid for complete or heuristic and constructive or perturbative algorithms)

- **State or (Candidate) solution**: a definition of the states of the search
- **Search Space**: A set of possible **states** that the search can be in.
- **Initial State**: State the search starts in.
- **Goal**: A set of one or more goal states. Sometimes there is one goal state sometimes there is a small set of alternative goal states
- **Evaluation function** $f(s)$: assesses the distance from a potential goal.
  (note: different from "objective", it can also include penalties due to constraint violations).

Components of a Search Algorithm (2)

- **Action Type** t: available to the algorithm. Neighborhood
- A finite set of **actions** of type t that can be executed in s, $\text{Actions}(t, s)$. Move
- A **transition model** that describes what each action does. $\text{Result}(s, a)$ returns the state that results from doing action $a \in \text{Actions}(t, s)$ in state s. Apply Move
- An **action-cost function**, $\text{Action-Cost}(s, a, s')$, that gives the numeric cost of applying action a in state s to reach state s'. Increment

ROAR-NET Application Programming Interface (API)

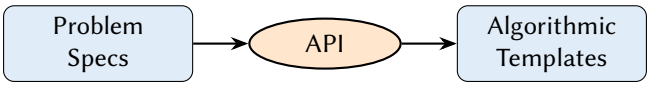
(Note: Here not meant as Web API, network-based API, or REST API.)

The ROAR-NET API Specification is the definition of an interface or protocol between optimization problems seen as black box and their solvers in order to facilitate understanding, reusing and scaling of solution approaches.

We look for a model that

- ... allows one to use off the shelf components to solve it.
- ... assumes a separation between **problem specifics** and **solver**.
- .. is designed as a **software interface** offering a service to other pieces of software and is implemented by the user.
- ... promotes **reusability** of software components and minimizes the user's effort to deploy a solution for the specific optimization problem at hand.
- ... maximizes code extensability, reusability, and simplicity.

The ROAR-NET API



- Standardized interface for **randomised optimisation algorithms**.
- Provides a set of types and methods to be **implemented by the user**, then used by the algorithms.

The ROAR-NET API

```
# User Types
Problem
Solution
Neighborhood
Move
Value
Increment
SubNeighborhood
```

```
# Methods of Problem
construction_neighbourhood
destruction_neighbourhood
local_neighbourhood
perturbation_neighbourhood
empty_solution
random_solution
heuristic_solution

# Methods of Solution
objective_value
lower_bound
copy_solution
```

```
# Methods of Neighborhood
moves
random_move
random_moves_without_replacement

# Methods of Move
lower_bound_increment
objective_value_increment
apply_move
revert_move
```

https://github.com/roar-net/roar-net-api-spec

Minimal API Functions Implemented

Problem

```
local_neighbourhood(Problem) : Neighbourhood
perturbation_neighbourhood(Problem) : Neighbourhood
random_solution(Problem) : Solution
```

Solution

```
objective_value(Solution) : Value[0..1]
copy(Solution) : Solution
```

SubNeighbourhood, Neighborhood, PerturbationNeighborhood

```
moves(Neighbourhood, Solution) : Move[0..*]
random_move(Neighbourhood, Solution) : Move[0..1]
```

Move

```
apply_move(Move, Solution) : Solution
revert_move(Move, Solution) : Solution
objective_value_increment(Move, Solution) : Increment[0..1]
```

Types

- **`Problem`** the problem instance
- **`Solution`** implements the representation of a **tentative** solution
- **`Value`** represents points in objective space
- **`Neighborhood`** a function that given a solution, gives another solution $neighbor : S \to S$, based on a neighborhood, compute a `Move`
- **`Move`** applied to a solution to get the novel solution
- **`Increment`** represents difference of `Value`

Simplification in single objective cases: **`Value`** and **`Increment`** are Reals (ie, **`Double`** or **`Integer`**)

Operations: Problem Representation

... on Problem (P):

- `empty_solution(P): Solution`
- `random_solution(P): Solution`
- `heuristic_solution(P): Solution[0..1]`

... on Solution S:

- `copy_solution(S): Solution`
- `lower_bound(S): Value[0..1]`
- `evaluation_value(): Value[0..1]`

Operations: Search

... on Neighborhood (N):

- `construction_neighbourhood(Problem): Neighbourhood`
- `destruction_neighbourhood(Problem): Neighbourhood`
- `local_neighbourhood(Problem): Neighbourhood`
- `moves(N, Solution): Move[0..*]`
- `random_move(N, Solution): Move[0..1]`
- `random_move_without_replacement(N, Solution): Move[0..*]`

Operations: Search

... on Move:

- `lower_bound_increment(Move, Solution): double[0..1]`
- `objective_value_increment(Move, Solution): double[0..1]`
- `apply_move(M, Solution): Solution` applies the move to a solution, to get a novel solution
- `revert_move(M): Move` computes the inverse of a move to revert it

Outline

Overview: Another Classification

- Solving Problems by Complete Search
  - Enumeration and Tree Search
  - Dynamic Programming
  - Backtracking (satisfaction problems)
  - Branch and Bound (optimization problems)
- Solving Problems by Incomplete Search
  - Incomplete Search Ideas
  - Construction-Based Metaheuristics

Recall: Complete Graph Search Methods

Tree (or graph) search in

Uninformed settings

- Breadth-first search
- Uniform-cost search
- Depth-first search

Informed settings

- Greedy best-first search
- A* search

Complete Tree Search

For CSPs with n variables and d values, we can use a complete tree search with

Search Space

Tree with branching factor at the top level nd
at the next level the branching factor is $(n-1)d$.
The tree has $n! \cdot d^n$ leaves even if only d^n possible complete assignments.

- CSP is **commutative** in the order of application of any given set of action (i.e., we reach same partial solution regardless of the order)
- Hence, generate successors by considering possible assignments for only a single variable at each node in the search tree. The tree has now d^n leaves
- use information: look-ahead, best first, etc.

Exploiting Information

Assessment of partial, infeasible solutions

The priority assigned to a node x is determined by the function

$$f(x) = g(x) + h(x)$$

$g(x)$: cost of the path so far
$h(x)$: heuristic estimate of the minimal cost to reach the goal from x.

- **greedy best-first search** uses h to decide
- **A* best-first search** uses f and is cost-optimal when reaches the goal, if $h(x)$ is an
  - admissible heuristic: never overestimates the cost to reach the goal
  - consistent: $h(n) \leq c(n, a, n') + h(n')$
    (consistent $\implies$ admissible, only necessary in graph search)

Best-First Search

```
Procedure TreeSearch(start, target)
add start to visited
add start to queue
while queue is not empty do
    current_node ← vertex of queue with min distance to target[a]
    remove current_node from queue
    for each neighbor n of current_node do
        if n not in visited then
            if n is target then
                return n
            else
                mark n as visited
                add n to queue
return failure
```

[a] Greedy best-first: min distance to target $= h(n)$;
A* best-first: min sum of distance so far and distance to target $= f(n)$

Heuristic Functions

Possible choices for admissible heuristic functions in $A^\star$ search:

- optimal solution to an easily solvable **relaxed problem**
- optimal solution to an easily solvable **subproblem**
- learning from experience by gathering statistics on state features
- preferred heuristic functions with higher values (provided they do not overestimate)
- if several heuristics available $h_1, h_2, \ldots, h_m$ and not clear which is the best then:

$$h(x) = \max\{h_1(x), \ldots, h_m(x)\}$$

A Star Search

Drawbacks:

- Time complexity: In the worst case, the number of nodes expanded is exponential,
  (but it is polynomial when the heuristic function h meets the following condition:

$$|h(x) - h^*(x)| \leq O(\log h^*(x))$$

  h^* is the optimal heuristic, the exact cost of getting from x to the goal.)

- Memory usage: In the worst case, it must remember an exponential number of nodes.
  Several variants: including iterative deepening A* (IDA*), memory-bounded A* (MA*) and simplified memory bounded A* (SMA*) and recursive best-first search (RBFS)